



Carrera **Desarrollo y Diseño Web**

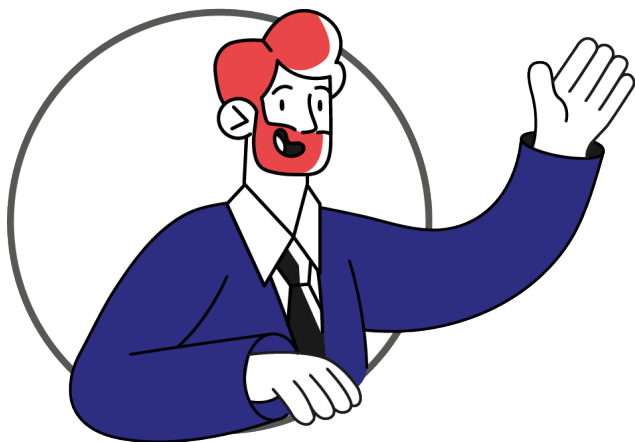
Experiencia 2 – Semana 6

**Modificando el contenido de una
página con JavaScript**

Índice

Introducción a la semana.....	3
Resultado de aprendizaje.....	4
Conceptos relevantes.....	5
Preguntas activadoras.....	5
Actividad.....	6
¿Qué son los eventos?.....	7
Event Listeners.....	10
Añadiendo un Event Listener.....	10
Quitando un Event Listener.....	15
¿Para qué usar los eventos?.....	18
Cierre de la semana.....	19
Referencias.....	20
Apuntes.....	21

Introducción a la semana



Entender cómo interactúan los usuarios con nuestras páginas web es esencial para crear experiencias interactivas y dinámicas. Por ello, Comprender cómo funcionan los eventos y cómo se manejan en JavaScript y HTML es esencial para desarrollar páginas web que respondan de manera efectiva a las acciones de los usuarios, mejorando así la experiencia del usuario y la funcionalidad del sitio web.

Durante esta semana, aprenderás qué son los eventos en el contexto de la programación web, cómo crearlos y usarlos, y cómo eliminarlos cuando ya no son necesarios. En este contexto, es relevante mencionar que podrás interactuar con los siguientes conceptos relevantes:

- **Función nombrada:** es una función con un nombre asociado que viene después de la palabra clave function (como “function suma(a, b)”).
- **Función anónima:** que, en lugar de tener un nombre después de la palabra clave function, la función se define sin un nombre en el lugar donde se necesita. Por ejemplo, “const resta = function(a, b) {}”
- **Declaración:** que es una instrucción que realiza una acción o define una entidad en el código.

Resultado de aprendizaje

El estudiante será capaz de:

RA1. Utiliza elementos básicos de programación en JavaScript en sitio web, de acuerdo criterios de usabilidad y accesibilidad y los requerimientos del proyecto.

RA2. Programa en JavaScript, considerando el control de elementos BOM (browser) y DOM (documento) para un sitio web bajo criterios de usabilidad, accesibilidad y estéticos.

Indicadores de logro:

IL1. Utiliza correctamente los elementos básicos de programación como variables, arrays, operadores (if - else) y bucles, a fin de dar solución a los requerimientos del proyecto.

IL2. Utiliza el lenguaje JavaScript para programar funciones que permiten reutilizar código de manera eficiente.

IL3. Distingue las funciones y aplicaciones del BOM y DOM en el proceso de desarrollo de un sitio web con JavaScript.

IL4. Utiliza JavaScript para seleccionar y controlar elementos del DOM de manera dinámica, para crear un sitio web que responda a las acciones del usuario.

IL5. Manipula los elementos del DOM, a fin de mejorar la usabilidad, accesibilidad y estética del sitio web.

IL6. Integra la selección, manipulación y control del DOM con el uso de matrices, bucles y funciones.

Conceptos relevantes

	Función nombrada	Función anónima	Declaración
	Eventos	Event Listeners	addEventListener

Preguntas activadoras

- ¿Cómo puedes hacer que tu página web reaccione dinámicamente a las acciones del usuario, como hacer clic en un botón o escribir en un formulario?
- Imagina que estás construyendo un formulario, ¿de qué manera asegurarías que este realice validaciones en tiempo real mientras el usuario escribe, sin tener que recargar la página?
- ¿Cómo podrías ejecutar una funcionalidad cuando el usuario haya interactuado con tu página de cierta manera?

Actividad

Descripción de la actividad

En la semana 6 realizarás la segunda actividad sumativa de la asignatura, que consiste en desarrollar una página web con el fin de incluir en ella comentarios, tal como harías en redes sociales, a través de un formulario HTML. Para ello, tendrás que integrar manejo de control de clase, agregar elementos a una caja, agregar contenido y utilizar JavaScript para manejar la funcionalidad del formulario, lo que implica leer los datos y agregar comentarios a la página.

¿Qué son los eventos?

En JavaScript, un evento es una acción o suceso que ocurre en el navegador y que puede ser detectado y manejado por el código. Estos eventos pueden ser desencadenados por diversas acciones, como la interacción del usuario, la carga de la página, o cambios en el estado de la aplicación. Los eventos permiten que el código responda dinámicamente a las acciones del usuario o a cambios en el entorno de la aplicación.

Algunos eventos comunes son:

- **click:** Ocurre cuando el usuario hace clic en un elemento, como un botón o un enlace.
- **change:** Se dispara cuando el valor de un elemento de formulario cambia. Por ejemplo, en un campo de entrada o un menú desplegable.
- **submit:** Ocurre cuando un formulario es enviado. Puede utilizarse para validar datos antes de enviarlos al servidor.
- **load:** Se dispara cuando la página web ha terminado de cargarse completamente.
- **keydown:** Se activa cuando una tecla del teclado es presionada.
- **keyup:** Se activa cuando una tecla del teclado es liberada.
- **mouseover:** Se disparan cuando el puntero del mouse está sobre un elemento.
- **mouseout:** Se disparan cuando el puntero del mouse deja de estar sobre un elemento.

Equivalencias

Para trabajar con eventos dentro de un botón, especialmente en el contexto de elementos HTML como botones, formularios y campos de entrada, generalmente se utiliza el prefijo "on" seguido del nombre del evento en minúsculas. Es así cómo se define qué acción debe realizarse cuando ocurre un determinado evento en el elemento. Por ejemplo, para el evento "click", utilizarías "onclick" en el botón.

A continuación, se presentan cada uno de los eventos JavaScript mencionados anteriormente y cómo se utilizan en un botón u otro elemento HTML:

Tabla 1*Equivalencias de eventos JavaScript.*

Evento JavaScript	Uso en un botón
Click()	Onclick Ejemplo: <code><button onclick="miFuncion()">Haz clic</button></code>
Change()	Onchange Ejemplo: <code><input type="text" onchange="miFuncion()"></code>
Submit()	Onsubmit Ejemplo: <code><form onsubmit="return validarFormulario()"></code> <code><!-- Contenido del formulario --></code> <code><button type="submit">Enviar</button></code> <code></form></code>
Load()	Onload Ejemplo: <code><body onload="iniciarPagina()"></code> <code><!-- Contenido de la página --></code> <code></body></code>
Keydown()	Onkeydown Ejemplo: <code><input type="text" onkeydown="verificarTecla(event)"></code>
Keyup()	Onkeyup Ejemplo: <code><input type="text" onkeyup="procesarEntrada()"></code>
Mouseover()	Onmouseover Ejemplo: <code><button onmouseover="mostrarMensaje()">Pasa el ratón</button></code>

Onmouseout()	Onmouseout Ejemplo: <button onmouseover="mostrarMensaje()">Pasa el ratón</button>
---------------------	---



Dato

La expresión “escuchar u observar eventos” se refiere al proceso de monitorear ciertas acciones o "eventos" que ocurren en una página web o en elementos de esta. Esto se hace para poder ejecutar código en respuesta a dichos eventos. En definitiva, "escuchar" es una metáfora que describe cómo el código se queda a la espera de que se produzca un evento para actuar en consecuencia.

Event Listeners

Para manejar los eventos, se utilizan oyentes de eventos u observadores (**event listeners**). Estos no son otra cosa que funciones que se asignan a un elemento para "escuchar" o detectar cuando ocurre un evento específico en él como un clic en un botón o una pulsación de tecla. Cada evento listener está encargado de responder a un evento específico, por lo que se definen individualmente para cada uno de los eventos que se quiera escuchar. Los oyentes pueden ser añadidos y eliminados dinámicamente, permitiendo que la lógica de la aplicación responda de manera adecuada a la interacción del usuario.

Añadiendo un Event Listener

Para agregar un event listener, se utiliza el método `addEventListener` que se aplica a un elemento del DOM. Este método permite especificar cómo debe responder ese elemento ante ciertos eventos, como clics, movimientos del mouse, cambios en un formulario, etc.

La sintaxis de **`addEventListener`** tiene la siguiente estructura:

```
elemento.addEventListener(evento, función);
```

Revisemos cada parte de esta estructura:

- **elemento:** es el elemento del DOM al cual se le desea agregar el event listener.
- **evento:** es una cadena que representa el tipo de evento al cual se está escuchando. Por ejemplo, **'click'**, **'change'**, **'mouseover'**, etc.
- **función:** es la función que se ejecutará cuando el evento ocurra. Puede ser una función nombrada o una función anónima. Esta función recibe un objeto Event como parámetro, que contiene información sobre el evento.

Ejemplos

A continuación, revisaremos distintos ejemplos relacionados con el uso de event listener en JavaScript para interactuar con el DOM y responder a las acciones del usuario.

Agregar un event listener a un botón en una página web

En la figura 1, **miBoton** es el elemento del DOM al cual se le está agregando el event listener. Puedes seleccionar el elemento de varias maneras, pero aquí se asume que el elemento tiene un ID específico llamado '**miBoton**'.

- Una vez que se tiene la referencia al elemento, puedes agregar un event listener a ese elemento. Esto se hace utilizando el método `addEventListener`.
- En este ejemplo, se define que el evento a escuchar es 'click'. Así, estamos definiendo qué vamos a escuchar en el evento de cambio, que ocurre cuando el valor del input cambia.
- Finalmente, definimos una función anónima que se ejecutará cuando ocurra el evento. En este caso, mostrará un mensaje de alerta con el texto: ¡Hiciste clic en el botón!

Figura 1

Ejemplo de `addEventListener`

```
<input type="button" id="miBoton" value="presióname">

<script>
  let boton = document.getElementById('miBoton');

  boton.addEventListener('click', function() {
    alert('¡Hiciste clic en el botón!');
  });
</script>
```

Input con Event Listener para 'change'

En el ejemplo de la figura 2 tenemos dos variaciones con respecto al ejemplo anterior: ahora trabajamos con un campo input que recibe textos escritos por el usuario y al declarar la función le pasamos un parámetro; en este caso, event.

Esta declaración establece un event listener en el campo de entrada de texto con ID miInput. Esto hace que escuchemos al campo 'miInput'. Además, se gatillará cuando su valor cambie. Finalmente, pasamos a la función el parámetro event, que contiene información sobre el evento en sí.



Dentro de la función anónima, se establece que se escribirá un texto en la consola con un valor fijo, y otro que será dinámico. El valor fijo es "Valor actual del input", mientras que el dinámico es obtenido del parámetro evento que se pasó anteriormente. Así, event.target se refiere al elemento que desencadenó el evento (en este caso, el input) y al agregar 'value' (**event.target.value**), se obtiene el valor actual del input cuando cambia.

Figura 2

Ejemplo de addEventListener con parámetro de evento.

```
<input type="text" id="miInput">

<script>
  // Seleccionar el elemento input por su ID
  let miInput = document.getElementById('miInput');

  // Añadir un event listener para el evento 'change'
  miInput.addEventListener('change', function(event) {
    // La función que se ejecutará cuando ocurra el evento 'change'
    console.log('Valor actual del input:', event.target.value);
  });
</script>
```

Enlace con Event Listener para 'click' y preventDefault

En este ejemplo, todo es igual a los ejemplos anteriores, excepto la línea que declara **evento.preventDefault()**. Este método impide que se lleve a cabo el comportamiento predeterminado del elemento. Cuando se llama a esta función dentro de un controlador de eventos, se detiene la acción predeterminada del evento.

En la figura 3, el elemento que estamos escuchando es un link, cuyo comportamiento normal es cargar la página que tiene definida. Al agregar el método **preventDefault**, se evita que el enlace siga su comportamiento predeterminado y, en su lugar, muestra un mensaje en la consola. En este caso, el mensaje **Enlace presionado**.

Figura 3

Ejemplo de addEventListener que bloquea el comportamiento predeterminado del evento.

```
<a href="http://www.duoc.cl" id="miEnlace">Ir a la página de DUOC</a>

<script>
  let enlace = document.getElementById('miEnlace');

  enlace.addEventListener('click', function(event) {
    event.preventDefault(); // Evita el comportamiento
                           //predeterminado del enlace
    console.log('Enlace presionado');
  });
</script>
```

Botón con Event Listener y función nombrada

Otra forma de definir la función a ejecutar se muestra en la figura 4. Consiste en definir la función y luego invocarla al declarar **addEventListener**. Nota que, al crear la función, se le pasa el parámetro **'event'**.

Figura 4

Ejemplo de addEventListener con una función existente

```
<input type="button" id="miBoton" value="Presioname">

<script>
  // Seleccionar el elemento button por su ID
  const miBoton = document.getElementById('miBoton');

  // Definir la función del event listener
  function handleClick(event) {
    console.log('¡Hiciste clic en el botón!');
  }

  // Añadir el event listener al botón
  miBoton.addEventListener('click ', handleClick);
</script>
```

Quitando un Event Listener

Para eliminar un event listener previamente añadido a un elemento del DOM, se usa el método **removeEventListener**. Esto es útil cuando deseas dejar de escuchar un evento específico en un momento determinado de la ejecución de tu código.

La sintaxis de **removeEventListener** tiene la siguiente estructura:

Figura 5

Sintaxis de removeEventListener

```
elemento.removeEventListener(evento, función);
```

Revisemos cada parte de esta estructura:

- **elemento:** es el elemento del DOM del cual deseas quitar el **event listener**.
- **evento:** es una cadena que representa el tipo de evento que se está dejando de escuchar. Debe ser el mismo tipo de evento que se usó al agregar el **event listener**.
- **función:** es la función que se usó al agregar el **event listener**. Debe ser la misma función que se quiere quitar.

Ejemplo: Uso de Event Listener y removeEventListener

Para comprender cómo se pueden añadir y luego eliminar los event listeners en elementos HTML usando JavaScript, revisemos el código de la figura 6 donde se utiliza un botón con un event listener, y luego se muestra cómo se puede eliminar.

Figura 6*Ejemplo de removeEventListener*

```
// Seleccionar el elemento button por su ID
let miBoton = document.getElementById('miBoton');

// Definir la función del event listener
function handleClick(event) {
    console.log('¡Hiciste clic en el botón!');
}

// Añadir el event listener al botón
miBoton.addEventListener('click', handleClick);

// Después de cierto tiempo, quitar el event listener
setTimeout(function() {
    miBoton.removeEventListener('click', handleClick);
    console.log('Se quitó el event listener del botón.');
```

Revisemos el código de la figura 6:

- **Selección del botón por su ID**

En este caso, el código comienza seleccionando un botón con el ID 'miBoton', utilizando document.getElementById('miBoton').

- **Definir función**

Luego, se define una función del event listener llamada handleClick. Esta se ejecutará cuando el botón sea presionado. En este caso, la función simplemente imprime un mensaje en la consola del navegador que dice "¡Hiciste clic en el botón!".

-

- **Agregar event listener al botón**

La llamada a `miBoton.addEventListener('click', handleClick);`` añade el event listener al botón seleccionado anteriormente. Esto configura el botón para que, cuando sea clickeado, se ejecute la función `handleClick`` definida en el paso anterior.

- **Eliminar event listener después de un retraso**

Después de 5 segundos (5000 milisegundos), se utiliza `removeEventListener` para quitar el event listener del botón previamente agregado.



Importante

Para quitar un event listener correctamente, la función que se pasa a `removeEventListener` debe ser la misma que se utilizó al agregarlo. Por esta razón, **es recomendable usar funciones nombradas** y no funciones anónimas para los event listeners que se planea eliminar más adelante.

¿Para qué usar los eventos?

Definir escuchas de eventos te permite controlar lo que ocurre cuando un usuario interactúa con tu página y, por tanto, decidir cómo debería responder tu página a las acciones del usuario.

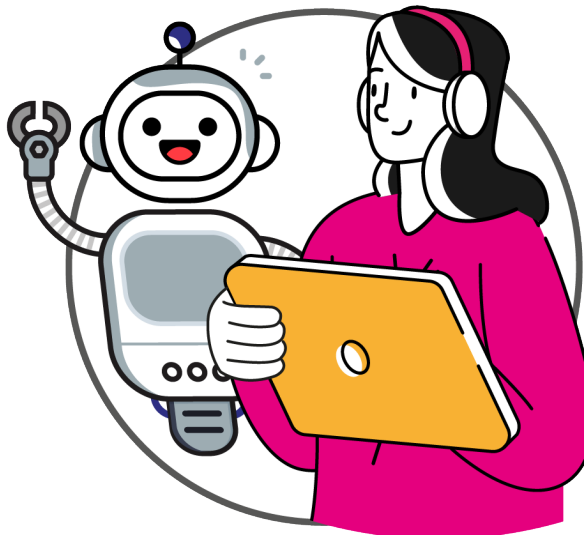
Algunas aplicaciones prácticas de esto pueden ser las siguientes:

- **Validación de formularios:** Al escuchar al evento **'submit'** puedes validar los datos ingresados antes de ser enviados. Si los datos están vacíos o son inválidos, puedes usar `preventDefault()` para impedir el envío.
- **Interacción con elementos de la interfaz de usuario (UI):** Cuando escuchas los eventos gatillados por acciones del usuario (tales como **'click'**, **'change'**, **'mouseover'**, o **'keydown'**, entre otros), puedes crear interactividad en botones, menús desplegables, etc. De esta forma, puedes cambiar el contenido o la forma de una página cuando el usuario realiza cualquiera de las acciones escuchadas.
- **Actualización dinámica de contenido:** Al poner una escucha en el evento **'load'**, puedes gatillar acciones que ocurran una vez que todo el contenido de la página se ha cargado. Así, por ejemplo, puedes obtener y mostrar datos desde un servidor después de que la página principal se haya cargado.

Cierre de la semana

Tal como viste durante esta semana, los eventos son de gran utilidad para generar reacciones en las páginas web. En esta oportunidad, has aprendido qué un evento es una acción o suceso que ocurre en el navegador y puede ser detectado y manejado por código JavaScript.

Conociste los eventos más comunes como click, change, submit, load, keydown, keyup, mouseover, y mouseout, y cómo cada uno se relaciona con las acciones del usuario o con cambios en el entorno de la aplicación. Además, aprendiste que el método `addEventListener` permite agregar un event listener, especificando el tipo de evento y la función que se ejecutará cuando dicho evento ocurra. Mientras que con `removeEventListener` es posible eliminar un event listener. En este caso, es importante recordar que la función que se pasa a `removeEventListener` debe ser la misma que se utilizó con `addEventListener`.



Referencias

- Ramírez, L. (Comp.). (2023) *JavaScript / 5 cosas que hay que saber*
<http://js.luisfel.cl>
- Mozilla Web Docs, Avanzado, Chile (s.f.). *Guía de JavaScript*
<https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Introducci%C3%B3n>

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.



DuocUC[®] ONLINE



Facilitador disciplinar: Claudio Navarro

Asesora par: Paola Veliz

Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.